

1. Apresentação

Vamos continuar o Projeto I, que abordou um sistema de atendimento usando vetor dinâmico e pilhas sobre tipos concretos, este projeto introduz tipos genéricos e o uso de estruturas encadeadas (lista duplamente encadeada, fila genérica e tabela hash).

O contexto é uma Biblioteca Digital, que permite explorar

- a lista duplamente encadeada para o acervo de livros (navegação nos dois sentidos);
- a fila genérica que gerencia a fila de espera por empréstimos;
- a tabela hash para indexar o catálogo para buscas rápidas por ISBN.

Conexão com o Projeto I: no projeto anterior você percebeu que `VetorDinamico` e `PilhaProcesso` precisariam ser reescritos para cada novo tipo de dado. Neste projeto você verá a solução: `No<T>`, `Fila<T>` e `NossoHash<K,V>` são genéricos e funcionam para qualquer tipo. A `ListaDupla`, propositalmente, ainda usa `int` — para que você sinta na prática a diferença e consolide a motivação para os genéricos.

2. O Problema

Uma biblioteca digital precisa de três mecanismos distintos para funcionar bem:

- Um acervo navegável: lista dos livros disponíveis que permita inserir e remover do início ou do fim, e percorrer nos dois sentidos (por exemplo, avançar e voltar páginas em uma interface de catálogo).
- Uma fila de espera: quando um livro está emprestado, os interessados entram em uma fila. O primeiro a solicitar é o primeiro a ser atendido quando o livro for devolvido (FIFO).
- Um catálogo indexado: para que o sistema localize qualquer livro instantaneamente pelo seu ISBN, sem precisar percorrer o acervo inteiro.

3. Estrutura do Projeto

O projeto é dividido em três etapas incrementais, cada uma introduzindo uma nova estrutura sobre as classes de domínio compartilhadas.

Etapa 1 — Classes de Domínio e Lista Duplamente Encadeada

Nesta etapa são construídas as classes que representam os dados do sistema e a primeira estrutura de armazenamento: a lista duplamente encadeada.

1.1 Classe Livro

Implemente a classe `Livro` com os seguintes atributos privados:

- `String isbn` — identificador único do livro (ex.: "978-85-430-0067-8").
- `String titulo` — título da obra.

- `String autor` — nome do autor.
- `int anoPub` — ano de publicação.
- `boolean disponivel` — true se o livro está disponível para empréstimo.

Requisitos da classe:

- Construtor recebendo todos os atributos; `disponivel` deve iniciar como true.
- Getters para todos os atributos e setters para `disponivel`, `titulo` e `autor`.
- Método `toString()` legível, por exemplo:
`[978-85-430-0067-8] Estruturas de Dados - Goodrich (2013) [DISPONIVEL]`
- Método `equals()` comparando livros pelo ISBN.

1.2 Classe Usuario

Implemente a classe `Usuario` com:

- `int matricula` — identificador único do usuário.
- `String nome` — nome completo.
- `String email` — endereço de e-mail.

Requisitos: construtor completo, getters para todos os atributos, setter para email, `toString()` legível e `equals()` por matrícula.

1.3 Classe NoDuplo e Classe ListaDupla

Implemente `NoDuplo` e `ListaDupla` para armazenar objetos do tipo `Livro` (tipo concreto, sem genéricos). A `ListaDupla` deve oferecer:

- `void insereInicio(Livro livro)` — insere no início da lista.
- `void insereFim(Livro livro)` — insere no final da lista.
- `Livro removePrimeiro()` — remove e retorna o primeiro livro; retorna null se vazia.
- `Livro removeUltimo()` — remove e retorna o último livro; retorna null se vazia.
- `Livro buscarPorIsbn(String isbn)` — percorre a lista e retorna o livro com o ISBN informado, ou null se não encontrado.
- `void listarDoInicio()` — exibe os livros do primeiro ao último.
- `void listarDoFim()` — exibe os livros do último ao primeiro, navegando pelos ponteiros anterior.
- `int tamanho()` — retorna o número de livros na lista.

Crie um programa de teste que demonstre todas as operações, incluindo a navegação nos dois sentidos e a busca por ISBN.

Etapa 2 — Fila Genérica e Gestão de Empréstimos

Com o acervo organizado, o sistema precisa gerenciar quem está esperando por um livro emprestado. A estrutura correta é a fila: o primeiro usuário a solicitar é o primeiro a ser atendido (FIFO).

2.1 Classe No<T>

Implemente a classe genérica No<T> com:

- Atributo privado `T info` — a informação armazenada no nó.
- Atributo privado `No<T> proximo` — referência para o próximo nó.
- Construtor, getters e setters para ambos os atributos.
- `toString()` retornando `[info]`.

2.2 Classe Fila<T>

Implemente a classe genérica Fila<T> usando No<T> internamente, com os atributos primeiro e ultimo. A fila deve oferecer:

- `void enqueue(T info)` — insere no final da fila (último a entrar, último a ser atendido).
- `T dequeue()` — remove e retorna o primeiro elemento; lança `FilaVaziaException` se vazia. Atenção: ao esvaziar a fila, atribua null a ultimo para evitar memory leak.
- `T primeiro()` — consulta o primeiro elemento sem removê-lo.
- `boolean filaVazia()` — retorna true se não há elementos.
- `int tamanho()` — retorna a quantidade de elementos.
- `toString()` — exibe todos os elementos no formato `[A] -> [B] -> [C] -> \\\`.

2.3 Classe GestorEmprestimos

Crie a classe GestorEmprestimos que coordena livros e filas de espera. Internamente use um `NossoHash<String, Fila<Usuario>>` (antecipando a Etapa 3) onde a chave é o ISBN do livro e o valor é a fila de usuários aguardando. Por enquanto implemente apenas:

- `void solicitarEmprestimo(String isbn, Usuario u)` — se o livro estiver disponível, marca-o como indisponível e registra o empréstimo; caso contrário, enqueue o usuário na fila de espera do livro.
- `void devolverLivro(String isbn)` — marca o livro como disponível; se houver usuários na fila de espera, dequeue o primeiro e registra o próximo empréstimo automaticamente.
- `void listarFilaDeEspera(String isbn)` — exibe a fila de espera de um livro.

Crie um programa de teste que simule: empréstimo de um livro disponível; tentativa de emprestar o mesmo livro (gerando fila); devolução (com atendimento automático do próximo da fila); e uma segunda devolução (com fila vazia).

Etapa 3 — Tabela Hash e Catálogo Indexado

A etapa final adiciona busca instantânea por ISBN ao sistema, usando uma tabela hash com dois parâmetros de tipo genérico.

3.1 Classe Entrada<K,V>

Implemente a classe Entrada<K,V> com:

- Atributos de pacote (sem modificador) `K key`, `V value` e `Entrada<K,V> proximo` (para encadeamento em caso de colisão).

- Construtor recebendo key e value; proximo inicia como null.

3.2 Classe NossoHash<K,V>

Implemente a classe NossoHash<K,V> usando um array interno de Entrada<K,V>[] com capacidade inicial 16 e encadeamento externo para tratar colisões. Métodos:

- `void put(K key, V value)` — calcula a posição pela função hash e insere no início da lista encadeada naquela posição (se a chave já existir, o novo valor é inserido à frente).
- `V get(K key)` — localiza e retorna o valor associado à chave; retorna null se não encontrado.
- `boolean containsKey(K key)` — verifica se a chave existe na tabela.
- `boolean containsValue(V value)` — verifica se o valor existe em qualquer posição da tabela (requer varredura completa — por que?)
- `void exibeMap()` — exibe todas as posições da tabela, inclusive as vazias, mostrando as cadeias de entradas em cada posição.

Use a função hash vista em aula.

3.3 Classe Catalogo

Crie a classe Catalogo que encapsula um NossoHash<String, Livro> (chave = ISBN, valor = Livro) e oferece:

- `void cadastrar(Livro livro)` — insere o livro no catálogo.
- `Livro buscar(String isbn)` — localiza e retorna o livro pelo ISBN.
- `boolean existe(String isbn)` — verifica se o ISBN está cadastrado.
- `void exibirCatalogo()` — exibe todos os livros cadastrados.

3.4 Integração Final — Sistema Completo

Integre as três etapas em uma classe BibliotecaDigital que usa simultaneamente:

- Um `Catalogo` (NossoHash<String, Livro>) para localização instantânea.
- Uma `ListaDupla` para o acervo navegável (inserção, remoção, listagem nos dois sentidos).
- Um `GestorEmprestimos` (Fila<Usuario>) para gerenciar a fila de espera.

Implemente um menu de console com as opções:

- Cadastrar livro (insere na ListaDupla e no Catalogo).
- Buscar livro por ISBN (usa o Catalogo).
- Listar acervo do início ao fim.
- Listar acervo do fim ao início.
- Solicitar empréstimo (usuário + ISBN).
- Devolver livro (ISBN).
- Ver fila de espera de um livro.
- Sair.